

COMPUTATION AND VERIFICATION REPORT

QIAN QI
Peking University

1. REVIEW OBJECT AND REPRODUCIBLE ENTRY POINTS

This report accompanies *Certified Bellman Operators for Costly Policy Revision*, revision R38. The reviewed source ancestry is identified in the response letter; the final publication manifest identifies the exact committed review object. The experiment definitions and tolerances are unchanged during publication. New tables are generated from committed machine-readable results, not from prose counts.

From the repository root, the scientific execution command is

```
pythonrevisions/2026-09-24-r38/replication/run_core.py
```

It executes all eight scientific stages, checks their return codes, verifies the headline counts, and compares 187 canonical exact objects with the recorded development execution. The publication command is

```
pythonrevisions/2026-09-24-r38/build/publish.py
```

It validates existing execution objects, generates tables, compiles the five root entry points, checks the resulting logs/PDFs, records historical-file preservation, and writes the completed publication manifest. A source snapshot or a workflow definition alone is not a completed review object.

2. INPUTS, OBJECTS, AND ARITHMETIC CONTRACTS

2.1. *Frozen primary inputs*

The primary cohort retains the seven R34 installed-rule families at each of three horizons and two operating tolerances. Neural fitting is not repeated. The raw-policy serialization and its SHA-256 are retained in every primary proof. All newly computed operating, implementation, and feasibility quantities are obtained from the explicit maintenance primitives rather than from an assumed correctness label on an old result. The inherited feasible R34 candidate is one attempted upper candidate and is reevaluated with the new exact policy evaluator.

The exact operating program is shared across the fourteen configurations at each horizon. Constructed L/U operating witnesses, exact necessary-action lower costs, safe-witness lower costs, and seven named upper attempts are evaluated with Fraction arithmetic. A policy is eligible as an upper bound only after its entire operating-regret function passes the all-restart test. The policy's cost uses its own transition law. Integrated equality and complete function equality are distinct Boolean fields.

2.2. *Primary proof objects and separate verification*

Each `results/primary/*.json.gz` object specifies rational knots, open-cell affine coefficients, isolated-point values, terminal conditions, actions, operating values, cost functions, witnesses, exact integrals, and hashes. Decimal table entries are derived renderings. Rational

strings are parsed as exact numbers; illegal actions, nonincreasing knots, inconsistent dates, malformed strings, or mismatched primitives fail verification.

The primary verifier is a separate SymPy implementation. It rebuilds the transition pullback partition and verifies every relevant open-cell endpoint limit and isolated point. It checks both Bellman equalities and inequalities, exact necessary-action membership, minimizing selectors, policy recursions, cost integrals, and exactness scope. It does not import the Fraction/PW implementation. Twenty corruption categories exercise the model, parser, partition, policy, numerical values, and metadata. The mutation ledger records each expected rejection rather than describing a test category without running it.

2.3. *Constrained frontier references*

The 70 fixed-restart frontier objects contain all recursively used history-conditioned feasible frontiers and the exact minimum initial cost. The verifier rebuilds successor candidate combinations in its own rational implementation and checks primal convex combinations and supporting lines. Candidate feasibility includes every downstream decision history, not only the initial operating constraint. Reusing a statewise frontier set does not force the selected continuation point to be the same after every history reaching that state.

The two analytic two-period uniform-initial randomized Markov references are separate entries in `results/frontier.json`. Their cost integrals reduce to rational terms and logarithms. The reported rational enclosures use an explicit 40-term logarithm series with a remainder bound. Those two analytic references are not included in the count of 70 independently checked finite-tree objects or in the 42 primary deterministic objects.

2.4. *Adaptive local LP*

The 12 local experiments vary the operating tolerance, the witness type, and the requested local accuracy. A fixed zero continuation support floor is used in this diagnostic. The constructor chooses a pointwise exact primal/dual pair, then proves feasibility and a gap bound over the entire affine input cell by rational polynomial sign tests. It validates dual-line changes and isolated points explicitly. Cells that fail are refined; a depth-cap event is an error rather than a certificate.

A preliminary unquantized local attempt encountered excessive rational-string growth. Its failure log is preserved in the history directory. The accompanying reconstruction source is labeled as a reconstruction, not as a byte-identical original run artifact. The published constructor uses 40-bit directed downward coefficients and includes the resulting analytic error allowance. The development execution subsequently added and passed a stronger full-dual/isolated-point validation; the publication execution reruns the complete final constructor and records its full time. The older local timing before that added validation is not used as the publication timing.

2.5. *Economic sensitivities*

Each of the 66 rational sensitivity objects retains the changed primitive, the installed rule, the common sufficient action restriction, both policy selectors, their separately evaluated operating/cost functions, and the exact initial-density integral. All zero savings remain in the output. The two sensitivity tolerances are explicitly separate from the unchanged primary tolerances. Scenario counts have no sampling interpretation.

TABLE II
CHARGED PRIMARY CONSTRUCTION WORK AND REPRESENTATION

Horizon	Operating DP (s)	Median revision (s)	Max. lower pieces	Max. bits
4	0.0046	0.0973	11	143
8	0.0141	0.3478	90	186
12	0.1198	2.3529	430	242

Notes. The operating DP is executed once per horizon. Revision time includes both exact-value and safe-witness outer bounds and the complete upper portfolio. Independent verification, support diagnostics, and supplementary studies are additional work, not omitted overhead. Timings are generated from this publication execution.

2.6. Nonlinear whole-box objects

The nonlinear experiment stores 20 witness-array sets and 76 candidate-policy records. All Bellman enclosures use outward integer rounding at scale 2^{34} . Monotone bilinear maps are bounded by box corners; the generated successor-cell spans are checked before the four-corner array stencil is used. Reward, terminal, and revision-indicator bounds cover complete boxes, including those crossed by an installed-rule threshold. Integer-range assertions reject an unsafe intermediate magnitude.

The pilot is a midpoint-compiled box action rule. It is not the exact continuous installed rule on boxes that cross an action threshold, and its implementation cost is consequently not assumed zero there. Restart and classical grid candidates are assessed with the same whole-box policy evaluator. A failed sufficient operating certificate does not identify an actual operating violation. The nonlinear arrays receive exact integer/read-only consistency checks, not the separate SymPy row/partition verification used for the one-dimensional primary cohort.

3. COMPLETE EXECUTED ACCOUNTING

TABLE I: Executed stage wall times

Stage	Seconds
Primary construction	51.2437
Primary independent check	195.4939
Economic sensitivities	7.4875
Fixed-restart frontiers	3.7650
Frontier independent check	7.9566
Adaptive local LP	273.9672
Nonlinear economy	46.0249
Global support diagnostics	130.6804

Notes. These are complete stage process times as recorded by the ledgers, with shared operating and witness work charged in the constructing stage. The run-level ledger separately records subprocess and dependency overhead. Historical proposal training is not included and is not claimed to have been rerun. The stage tables charge the fresh construction and verification work. The run-level execution ledger also records subprocess wall times, Python/platform information, and return codes. Package installation and PDF production are publication overhead, recorded in the workflow/build logs rather than hidden in an optimization timing. Exact operating DP is not a comparator for the constrained revision objective merely because its runtime is small.

TABLE III
DETERMINISTIC OPTIMALITY ON THE UNCHANGED PRIMARY COHORT

Horizon	Cases	Exact initial	Positive exact	All-restart exact	Max. gap
4	14	14	10	14	0.000000
8	14	6	2	5	0.103473
12	14	4	0	4	0.248472

Notes. Exact initial means equality of rational uniform-initial costs. All-restart exact additionally means equality of every stored cost-to-go function. Both operating tolerances are included. The gap is a deterministic cost interval, not a randomized duality gap.

TABLE IV
SEPARATE CONTROLLED IMPROVEMENT COORDINATES

Coordinate	Strict improvements	Largest gain
Support witness	30	0.00008591
Global price refinement	28	0.31597630
Deterministic witness	30	0.00001170
Feasible upper enrichment	30	0.93987965

Notes. Each row compares identical initial distributions and installed rules within its own policy-class scope. Gains across rows are not additive. Global prices expand from eight to fifteen; the local LP study has its own zero floor.

The serialized primary fields report piece counts and rational bit lengths. The local-LP fields report accepted cell counts and the exact propagated error. Nonlinear records report grid/horizon sizes, allocated array storage, and actual candidate times. These are representation and runtime diagnostics, not a proof of polynomial or dimension-free complexity. The supplemental output contains every row, including failed certificates.

4. RESULT INVENTORY AND THE LIMITS OF AUDIT INDEPENDENCE

There are 42 primary proof objects, 66 sensitivity objects, 70 fixed-restart frontier objects, 12 adaptive local-LP objects, 21 global-support bundles, and 42 gap-function bundles. The nonlinear archive separately stores witness and policy arrays. The canonical cross-execution comparison covers 187 objects in the primary, frontier, local-LP, support, and gap-function directories. It excludes timing ledgers and NPZ container timestamps; it is an exact object-identity check, not a second proof system.

Mathematical implementation independence is narrower and explicit. Forty-two primary objects and 70 frontier objects pass the separate SymPy verifier. Local-LP, sensitivity, and nonlinear checks do not acquire that label because they appear in the same publication. The independent checker and constructor still share the intended mathematical model, file format, and general Python execution environment. Mutation tests reduce specified implementation risks but do not prove freedom from all bugs.

5. GAP AND POLICY-CLASS LEDGER

The full primary exact-value index contains the exact lower/upper fractions, objective and policy-class strings, integrated/functionwise flags, policy hash, lower-function hash, raw-rule hash, and proof path/hash. The maximum deterministic gap is not reported as a randomized duality gap. The 70 Dirac-history objectives are not averaged into a uniform Markov optimum.

The nonlinear tolerance of $1/2$ does not replace 0.01 or 0.05 in the primary cohort. Exact arithmetic certifies the declared finite object, not an unstated extension of its policy class or economic model.

6. PUBLICATION AND PRESERVATION CHECKS

The science workflow commits its actual execution outputs to the isolated R38 branch. The publication workflow starts from that committed state, materializes the authored paper source, rebuilds every generated table, and compiles the five root PDF objects. Every final log is checked for undefined references/citations, fatal errors, and materially overfull boxes. PDF metadata and extracted text are inspected programmatically; representative full-page renders are inspected separately during document preparation. The historical annex incorporates the earlier R34 article, supplement, response, computation report, and recursive history PDF without deleting their pages.

The preservation manifest checks that files inherited from the review base remain unchanged, except the root revision index whose exact former content is retained under the R38 history directory. Old review reports, protocols, source, failed runs, and manuscript PDFs are not overwritten. New scientific and publication files live in the R38 tree or use new R38 root names. No force-push or modification of a different branch is part of the publication procedure.

The publication manifest deliberately records the source commit and workflow identifier rather than attempting a self-referential hash of its own final Git commit. The final remote branch HEAD identifies the committed package containing that manifest. Its file-hash map covers the completed new artifacts; the external Git commit gives the authoritative review snapshot. A failed run retains its logs and is not described as a passing publication.